Zadanie nr 3 - perceptron wielowarstwowy 4-2-4

Inteligentna Analiza Danych

Łukasz Centkowski 264303
Maciej Dominiak 264307

25 października 2026

1 Cel zadania

Zadanie polegało na wytrenowaniu perceptronu wielowarstwowego o architekturze 4-2-4 tak, aby kopiował swoje wejścia na wyjścia. Sprawdziliśmy przy tym, czy trening jest w ogóle możliwy, gdy neurony warstwy wyjściowej nie mają wejść obciążających, oraz dlaczego tak się dzieje.

2 Wstęp teoretyczny

Sieć ma trzy warstwy: cztery neurony kopiujące na wejściu, dwa liniowe bez biasu w warstwie ukrytej i cztery sigmoidalne z biasem na wyjściu. Razem 20 parametrów, czyli 16 wag i 4 biasy. Trening to wsteczna propagacja błędu [2], czyli gradientowa minimalizacja funkcji

$$E(\mathbf{w}) \;=\; \tfrac{1}{2}\sum_{\mu}\sum_{i}\bigl(z_i^{(\mu)} - y_i^{(\mu)}\bigr)^2. \tag{1}$$

Zatrzymuje ją wyłącznie zadana liczba epok L, bez progu błędu [1, 3].

Kluczowe jest to, co robi pojedynczy neuron wyjściowy. Dostaje z warstwy ukrytej dwie liczby, czyli *punkt na płaszczyźnie*, i ma dać wartość bliską 1 dla swojego wzorca, a bliską 0 dla trzech pozostałych. Jego granica decyzyjna to prosta

$$\mathbf{u}\cdot\mathbf{h} + b = 0, \tag{2}$$

gdzie $\mathbf{u}$ to wagi, a b bias. Bez biasu zostaje $\mathbf{u}\cdot\mathbf{h} = 0$, a taka prosta **zawsze przechodzi przez punkt** $(0, 0)$.

3 Eksperymenty i wyniki

Zbiór treningowy to cztery wzorce autoasocjacji: $\mathbf{x} = \mathbf{z} = [1,0,0,0]$, $[0,1,0,0]$, $[0,0,1,0]$ oraz $[0,0,0,1]$. We wszystkich eksperymentach użyliśmy kroku uczącego 0,5 i 20000 epok, a każdy wariant powtórzyliśmy 10 razy od różnych wag losowych z przedziału $[-1, 1]$.

3.1 Eksperyment nr 1: trening z wejściami obciążającymi

3.1.1 Założenia

Architektura dokładnie taka, jak w treści zadania: neurony wyjściowe mają wejścia obciążające.

3.1.2 Przebieg

Dziesięć przebiegów od różnych wag startowych. Po każdej epoce notowaliśmy wartość funkcji (1).

3.1.3 Rezultat

Błąd końcowy wyniósł od 0,000116 do 0,000122, czyli we wszystkich przebiegach praktycznie tyle samo. Wyjścia najlepszego przebiegu zawiera Tab. 1.

Tabela 1: Wyjścia sieci z wejściami obciążającymi

Wzorzec	y_1	y_2	y_3	y_4
1	**0,998**	0,005	0,000	0,007
2	0,005	**0,998**	0,007	0,000
3	0,000	0,003	**0,997**	0,005
4	0,003	0,000	0,004	**0,996**

Sieć odtwarza wszystkie cztery wzorce. Właściwe wyjście ma wartość bliską jedności, a pozostałe trzy są wygaszone poniżej 0,01.

3.2 Eksperyment nr 2: trening bez wejść obciążających

3.2.1 Założenia

Ta sama architektura i te same parametry, ale neurony wyjściowe pozbawione biasów. Liczba parametrów spada z 20 do 16.

3.2.2 Przebieg

Jak w eksperymencie nr 1. Dodatkowo sprawdziliśmy, czy wysoki błąd nie wynika ze zbyt krótkiego treningu albo ze źle dobranego kroku uczącego. Trening bez biasów powtórzyliśmy więc dla trzech wartości kroku uczącego (0,1; 0,5 i 2,0) i dla dwóch liczb epok (20000 i 50000). Daje to sześć kombinacji. Każdą uruchomiliśmy 5 razy od różnych wag startowych i zapisaliśmy najmniejszy uzyskany błąd. Wyniki zawiera Tab. 3.

3.2.3 Rezultat

Błąd końcowy wyniósł od 0,384495 do 0,444623, a więc ponad 3300 razy więcej niż z biasami. Wyjścia najlepszego przebiegu zawiera Tab. 2.

Tabela 2: Wyjścia sieci bez wejść obciążających

Wzorzec	y_1	y_2	y_3	y_4	
1	**1,000**	0,166	0,154	0,582	źle, fałszywe y_4
2	0,000	**1,000**	0,018	0,016	dobrze
3	0,019	0,151	**1,000**	0,000	dobrze
4	0,052	0,570	0,000	**0,834**	źle, fałszywe y_2

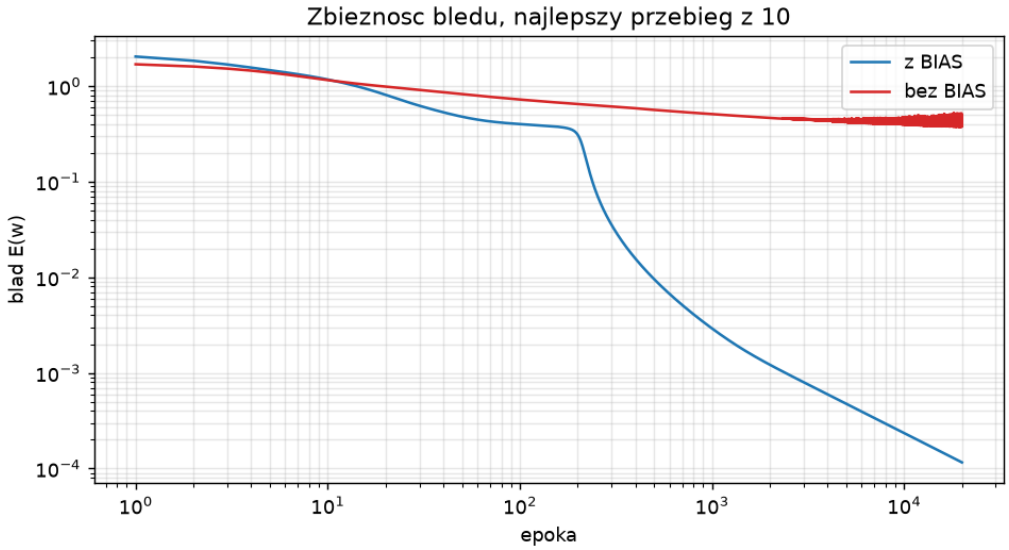
Dwa wzorce sieć odtwarza czysto, a przy dwóch pozostałych zapala wprawdzie właściwe wyjście, ale *nie potrafi wygasić fałszywego*: $y_4 = 0{,}582$ przy wzorcu 1 i $y_2 = 0{,}570$ przy wzorcu 4, choć oba powinny wynosić zero. Stąd bierze się cały błąd resztkowy.

Wynik nie jest skutkiem zbyt krótkiego treningu, co pokazuje Tab. 3.

Tabela 3: Najlepszy błąd bez biasów przy różnych parametrach treningu

Krok uczący	**Epoki**	**Najlepszy błąd**
0,1	20000	0,420717
0,1	50000	0,381481
0,5	20000	0,384495
0,5	50000	0,396724
2,0	20000	0,634560
2,0	50000	0,631445

Ani wydłużenie treningu, ani zmiana kroku uczącego nie zbijają błędu poniżej około 0,38. To jest faktyczna granica, a nie niedouczenie. Porównanie przebiegu zbieżności obu wariantów przedstawia Rys. 1.



Rysunek 1: Zbieżność błędu w najlepszym z dziesięciu przebiegów

3.3 Eksperyment nr 3: geometria warstwy ukrytej

3.3.1 Założenia

Warstwa ukryta ma dwa neurony, więc każdy wzorzec staje się punktem na płaszczyźnie. Chcieliśmy sprawdzić, ile z czterech punktów da się oddzielić prostą od pozostałych trzech, osobno dla prostej dowolnej i dla prostej przechodzącej przez początek układu.

3.3.2 Przebieg

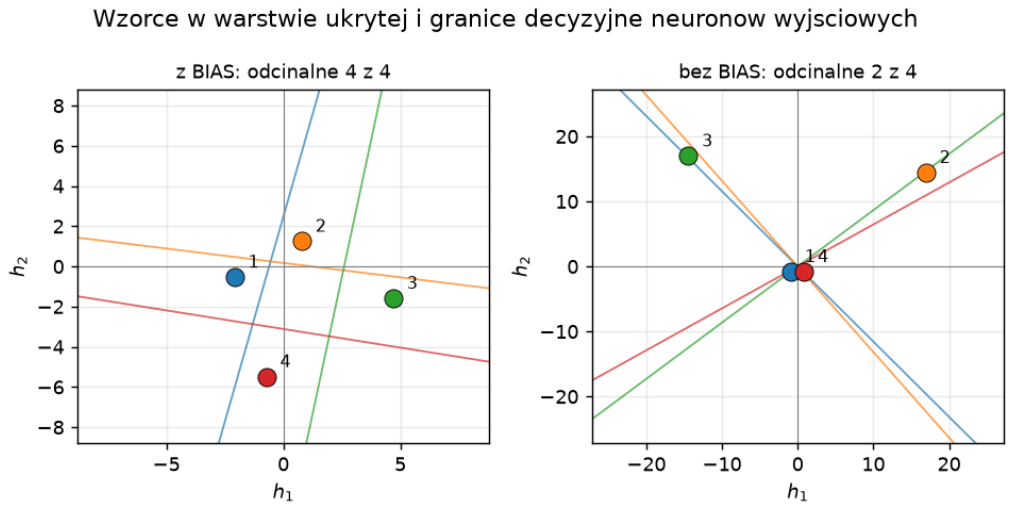
Dla najlepszego przebiegu każdego wariantu odczytaliśmy położenia czterech wzorców w warstwie ukrytej oraz granice decyzyjne wyuczone przez neurony wyjściowe. Następnie policzyliśmy, ile punktów da się odciąć każdym z dwóch rodzajów prostej.

3.3.3 Rezultat

Położenia wzorców zawiera Tab. 4, a obraz całości Rys. 2.

Tabela 4: Położenia wzorców w warstwie ukrytej

	z BIAS		**bez BIAS**	
Wzorzec	h_1	h_2	h_1	h_2
1	−2,075	−0,535	−0,834	−0,820
2	0,800	1,251	16,964	14,435
3	4,707	−1,591	−14,444	16,973
4	−0,713	−5,492	0,865	−0,806



Rysunek 2: Wzorce w warstwie ukrytej i granice decyzyjne neuronów wyjściowych

Wynik jest jednoznaczny i taki sam dla obu wariantów: prostą dowolną da się odciąć **4 z 4** punktów, a prostą przechodzącą przez początek układu tylko **2 z 4**. Nie zależy to od przebiegu treningu, bo jest własnością czterech punktów na płaszczyźnie, a nie konkretnych wag.

I to zgadza się z pomiarem: bez biasów poprawne są dokładnie dwa wzorce, bo tylko dwa neurony wyjściowe są w stanie wykonać swoje zadanie.

Na Rys. 2 widać też, jak sieć bez biasów próbowała sobie poradzić. Wszystkie cztery granice przechodzą przez punkt $(0, 0)$, więc sieć wypchnęła wzorce 2 i 3 bardzo daleko od środka, aż do wartości rzędu 17, gdzie prosta przez początek układu potrafi je oddzielić. Wzorce 1 i 4 zostały natomiast tuż przy początku, praktycznie jeden na drugim, i tam żadna taka prosta ich nie rozdzieli.

4 Wnioski

Czy perceptron dałoby się wytrenować bez wejść obciążających? Nie. Błąd zatrzymuje się na poziomie około 0,38 wobec 0,00012 przy sieci z biasami, czyli ponad 3300 razy wyżej, i nie spada mimo wydłużania treningu ani zmiany kroku uczącego. Dokładnie dwa z czterech wzorców są odtwarzane poprawnie; w pozostałych dwóch sieć zapala właściwe wyjście, ale nie potrafi wygasić fałszywego, które zostaje na poziomie 0,57–0,58 zamiast zera.

Jaka jest rola wejść obciążających? Bias daje granicy decyzyjnej **swobodę położenia**. Bez niego suma ważona wynosi $\mathbf{u} \cdot \mathbf{h}$, więc granica $\mathbf{u} \cdot \mathbf{h} = 0$ zawsze przechodzi przez początek układu współrzędnych. Z biasem jest to pełna postać (2), czyli prosta o tym samym nachyleniu, ale przesunięta tam, gdzie trzeba.

Najprostsza analogia: $\mathbf{u} \cdot \mathbf{h}$ to prosta $y = ax$, a $\mathbf{u} \cdot \mathbf{h} + b$ to $y = ax + b$. Prostej przechodzącej przez zero nie da się dopasować do danych, które wymagają wyrazu wolnego, i nie pomoże tu żadne dobieranie nachylenia.

W tym zadaniu konsekwencja jest policzalna. Cztery punkty w warstwie ukrytej tworzą czworokąt wypukły, więc *każdy* z nich da się odciąć jakąś prostą. Ale prostych przechodzących przez początek układu wystarcza tylko dla *dwóch* z nich, bo dla pozostałych prosta musiałaby przejść „pomiędzy", a przez zero tego nie zrobi. Stąd dwa poprawne wzorce zamiast czterech.

Drugi, bardziej obrazowy sposób: bez biasu zerowe wejście daje zerową sumę ważoną, a więc $f(0) = 0{,}5$ na każdym wyjściu. Sieć nie ma wtedy żadnego sposobu, by mieć jakiekolwiek „domyślne nastawienie". Bias jest właśnie tym nastawieniem: mówi neuronowi, jak bardzo ma być skłonny się wzbudzić, zanim jeszcze zobaczy jakiekolwiek wejście.

5 Załączniki

Konfiguracja sieci o architekturze z punktu 2.2 (plik `mlp.py`):

```
net = Network([4, 2, 4], bias=[False, use_bias],
              activations=[(linear, linear_derivative),
                           (sigmoid, sigmoid_derivative)])
```

Aktywacja i bias zadawane są osobno dla każdej warstwy: warstwa ukryta jest liniowa i bez biasu, wyjściowa sigmoidalna z biasem.

Bibliografia

[1] K. Stokfiszewski. *Inteligentna Analiza Danych. Materiały do wykładu nr 2*. Instytut Informatyki Politechniki Łódzkiej.

[2] D. E. Rumelhart, G. E. Hinton, R. J. Williams. *Learning representations by back-propagating errors*. Nature 323, 1986.

[3] S. Osowski. *Sieci neuronowe do przetwarzania informacji*. Oficyna Wydawnicza Politechniki Warszawskiej, Warszawa.